Procesamiento automatizado de facturas con n8n

Recibir, validar, guardar y recuperarse de errores sin revisar cada factura a mano

Equipo_JuanMunoz

Integrante: Juan Esteban Muñoz Díaz (diseño, construcción, pruebas y documentación)

Curso: Automatización inteligente de procesos con n8n · EAM · 2026 II · Segundo corte

Actualizado: 26 de octubre de 2026 · Versión: workflow «facturas-21/10»

Beneficia a: áreas de contabilidad o personas que reciben y registran facturas de clientes.

Problema y contexto

Una situación concreta

- Una persona de contabilidad recibe facturas por **correo, papel o desde el sistema de un proveedor**.
- Para cada una revisa que traiga **número, cliente, monto y estado**.
- Si falta algo, escribe al emisor y espera respuesta.
- Luego anota el dato **a mano** en una hoja.
- Resultado: trabajo repetitivo, **errores de digitación** y **facturas registradas dos veces**; y si algo falla, no queda registro de cuándo ni por qué.

Términos que se usan en esta presentación

n8n: herramienta que conecta pasos y servicios mediante workflows (flujos de trabajo) visuales.

Webhook: dirección web que n8n deja «escuchando»; otro sistema le envía una factura con un POST.

API: forma en que un programa pide o guarda datos en otro servicio (aquí, Baserow).

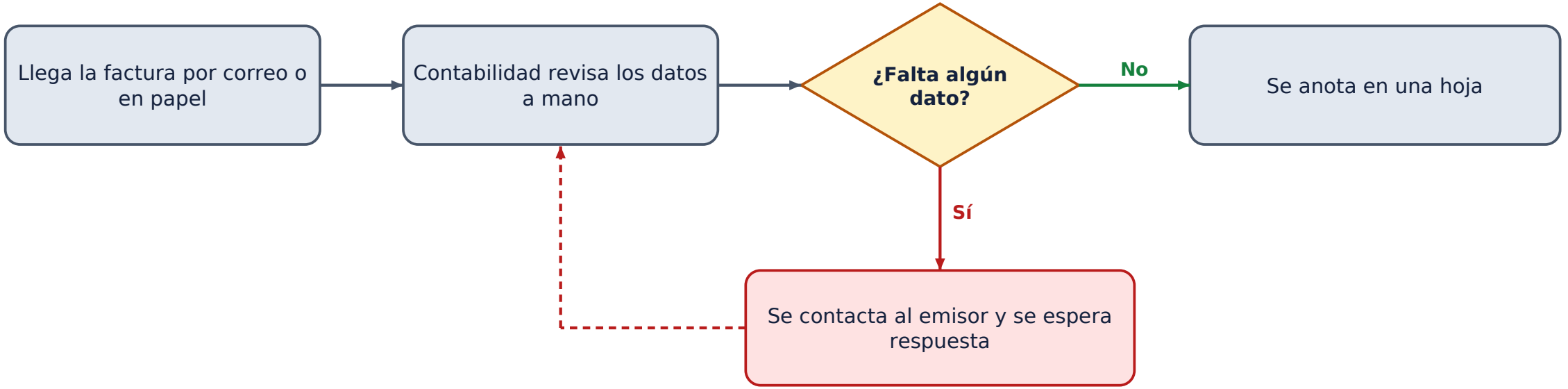
HTTP Request: nodo de n8n que llama a una API indicando método, dirección y datos.

Subworkflow: workflow pequeño y reutilizable que otro workflow llama para una tarea.

Baserow: base de datos en línea donde se guardan las facturas e incidentes.

Proceso manual (antes)

¿Cómo se trabaja hoy?



Dónde aparecen los problemas

- **Demoras:** cada factura depende de que una persona la revise y espere la respuesta del emisor.
- **Tareas repetitivas:** el mismo dato se digita varias veces.
- **Errores:** no hay control de duplicados; un error de digitación pasa desapercibido.
- **Sin trazabilidad:** no queda registro de cuándo ni por qué falló algo.

Objetivo y alcance de esta versión

Objetivo. Recibir una factura (formulario o desde otro sistema por HTTP), validarla, limpiar sus datos, evitar duplicados, guardarla en Baserow y responder al emisor **lo que realmente ocurrió**. Si falla un servicio técnico, dejar un incidente registrado y avisar.

Incluye

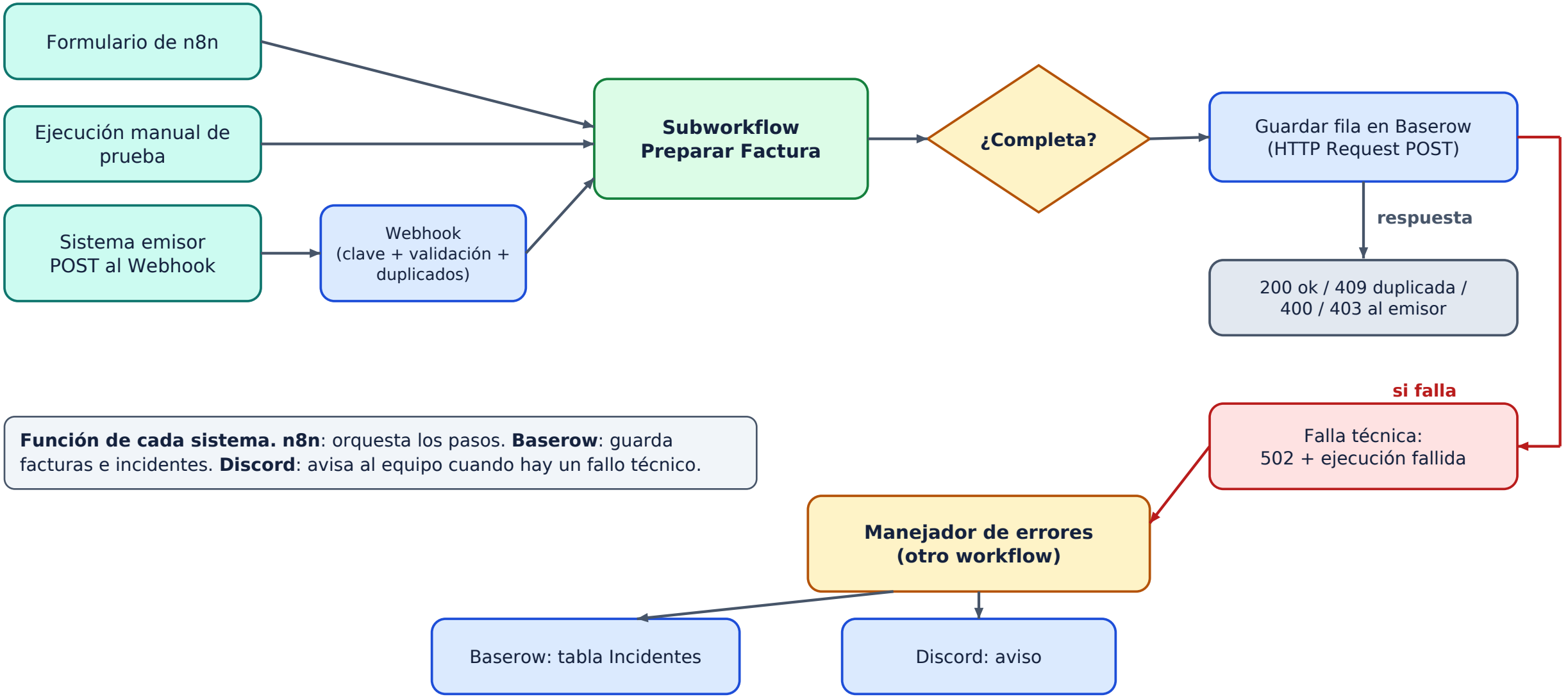
- Tres entradas: formulario de n8n, ejecución manual de prueba y **Webhook con clave**.
- Validación de 4 campos obligatorios y control de duplicados (en el Webhook).
- Subworkflow que limpia estado y cliente.
- Guardado en Baserow con HTTP Request.
- Errores: respuesta 502 honesta, ejecución fallida, manejador, tabla de incidentes y aviso a Discord.

Queda por fuera

- Lectura automática de PDF, imágenes o correos (OCR).
- Reintentos automáticos de guardado.
- Unicidad garantizada en la base de datos.
- Túnel o servidor público permanente.
- Validación contable (monto positivo, cliente existente).

Solución automatizada

Diagrama general



Función de cada sistema. n8n: orquesta los pasos. **Baserow**: guarda facturas e incidentes. **Discord**: avisa al equipo cuando hay un fallo técnico.

Recorrido de una factura completa

Caso: entrada por Webhook

1 · Entra por POST (datos ficticios)

```
{ "numero_factura": "C08-001",
  "cliente": "Cliente Ficticio",
  "monto": 150000,
  "estado": " Completa " }
```

Resultado obtenido

- Respuesta 200 con `{"ok":true}`.
- Una fila nueva en la tabla **Facturas** de Baserow, con cliente y estado ya limpios (`completa`).
- Al repetir el mismo envío: 409 y ninguna fila nueva.

2 · Clave correcta

Header Auth. Sin clave → 403 y no se ejecuta nada.

3 · Campos completos

Los 4 campos existen. Si falta uno → 400 sin tocar Baserow.

4 · Duplicado

Se consulta Baserow por numero_factura. **count = 0** → es nueva (si no → 409).

5 · Limpieza (subworkflow)

`" Completa "` → `"completa"`; cliente sin espacios sobrantes.

6 · Decisión

estado_limpio = completa → rama «completa».

7 · Guardado y respuesta

POST a Baserow crea la fila; el emisor recibe **200 ok:true**.

Integraciones

Qué intercambian los servicios y por qué

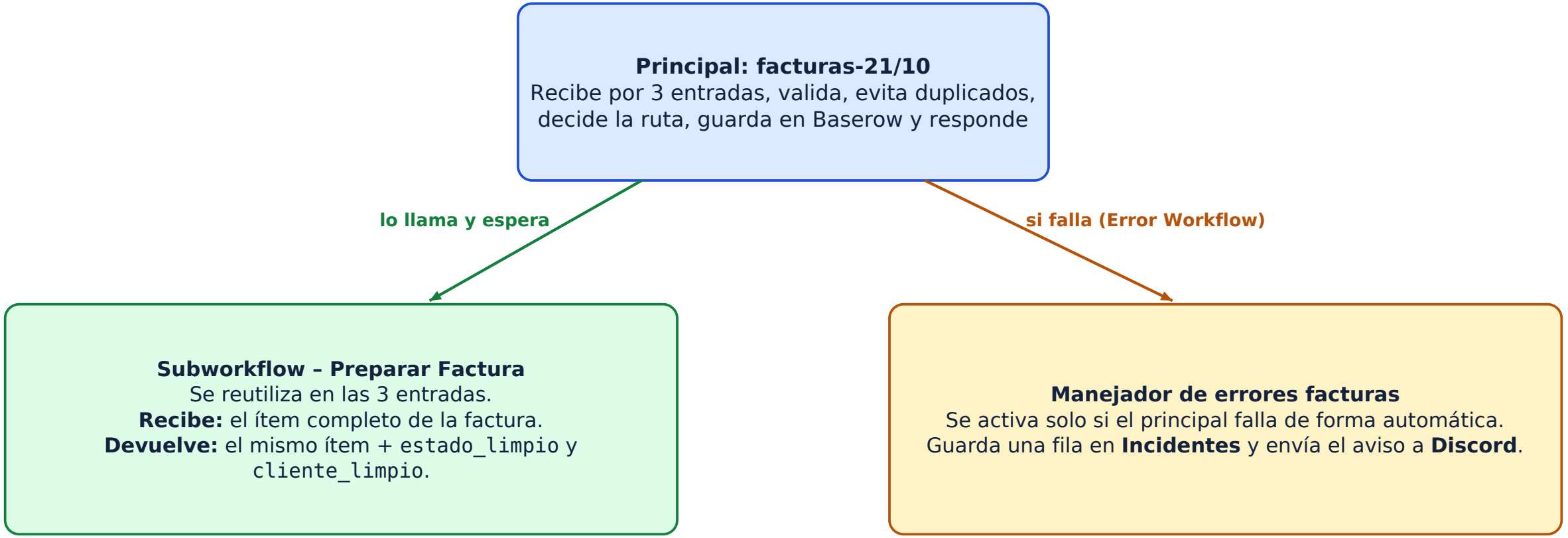
Servicio	Función	Cómo se conecta	Qué intercambia
Baserow	Base de datos: tablas Facturas e Incidentes	**HTTP Request** (3 nodos) con credencial Header Auth	Lee (GET) si la factura ya existe; escribe (POST) la factura y los incidentes
Discord	Avisar al equipo de un fallo técnico	Nodo nativo de Discord (webhook de canal)	Envía el mensaje con workflow, nodo y ejecución
Sistema emisor	Origen de las facturas	Webhook de n8n (POST + clave)	Entrega los 4 campos; recibe un código HTTP

Nodo HTTP Request	Método	Qué envía	Dato relevante de la respuesta
Buscar Duplicado	GET	`filter__numero_factura__equal` y `user_field_names=true`	`count`: filas que coinciden
Guardar en Baserow	POST	numero_factura, cliente (limpio), monto, estado (limpio), detalle	Fila creada con su `id`; si falla, sale por la salida de error
Guardar Incidente en Baserow	POST	fecha, workflow, nodo, mensaje, ejecucion, estado_atencion	Fila creada en Incidentes

Endpoint de Baserow: `https://api.baserow.io/api/database/rows/table/{ID_TABLA}/` — sin secretos; el ID es el de la tabla propia. No hay paginación porque solo se lee count de un filtro exacto.

Organización de los workflows

Tres archivos JSON que trabajan juntos



El principal **espera** al subworkflow porque necesita estado_limpio para decidir la ruta. El manejador se asocia al principal en Settings → Error Workflow. Orden de importación: subworkflow → manejador → principal.

El Webhook: cómo se prueba y qué responde

Aspecto	Detalle
Método y ruta	`POST /webhook/factura-c08` (producción) · `/webhook-test/factura-c08` (prueba)
Autenticación	Header Auth: encabezado propio (en las pruebas, `x-api-key`) con una clave definida por quien configura
Cuerpo esperado	JSON con `numero_factura`, `cliente`, `monto`, `estado`
URL de prueba	Responde solo mientras el editor está escuchando («Listen for test event»)
URL de producción	Responde solo si el workflow está **publicado/activo**

Código	Significado	Tipo
200	`ok:true` — factura guardada	Éxito
400	Faltan campos obligatorios	Negocio
403	Sin clave o clave incorrecta	Seguridad
409	Factura duplicada, no se crea otra fila	Negocio
502	`ok:false` — no se pudo guardar en Baserow	Error técnico

Sin túnel: el Webhook se probó localmente. Un túnel (cloudflared) sería **propuesto, no implementado**.

Validaciones y errores

No es lo mismo una respuesta del negocio que un fallo técnico

Respuesta del negocio

El sistema funciona y la regla no se cumple. **No activa el manejador.**

- **Faltan datos** → 400, se rechaza antes de escribir.
- **Clave incorrecta** → 403, no se ejecuta el flujo.
- **Factura ya registrada** → 409, no se crea otra fila.
- Ejemplo: `cliente` ausente → mensaje de campos obligatorios y Baserow sin cambios.

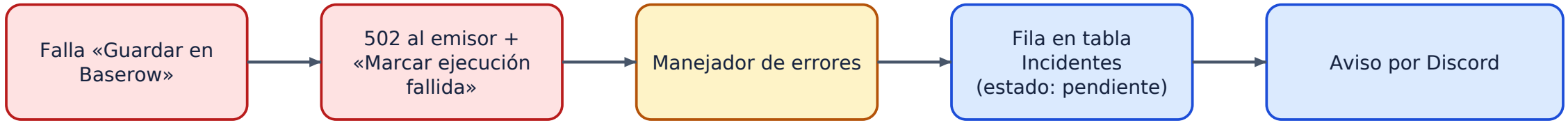
Error técnico

Un servicio no puede completar la operación. **Sí activa el manejador.**

- Baserow no responde, el token es inválido o la tabla no existe.
- El nodo «Guardar en Baserow» sale por su **salida de error**.
- Se responde `502 ok:false`: el emisor **nunca recibe un éxito falso**.
- «Stop and Error» marca la ejecución como fallida; sin eso el manejador no se activaría.

Limitación conocida: la validación trata `monto = 0` como faltante y no comprueba que sea numérico ni positivo.

Registro del fallo y recuperación



Cómo se recupera una operación

- **Corregir la causa:** tabla, token, permisos o disponibilidad.
- **Reenviar** la factura que no se guardó.
- **Sin duplicar:** antes de crear, el flujo consulta por numero_factura; si existe, responde 409.
- Pasar el incidente a resuelto a mano, tras verificar el reenvío.

Decisiones y límites

- **Sin reintentos automáticos**, a propósito: una respuesta perdida no prueba que no se guardó y reintentar a ciegas podría duplicar.
- «Preparar Incidente» solo arma datos; lo que guarda es «Guardar Incidente en Baserow».
- El Error Trigger solo actúa con ejecuciones **automáticas**, no manuales.
- Riesgo abierto: dos envíos simultáneos de la misma factura podrían crear dos filas.

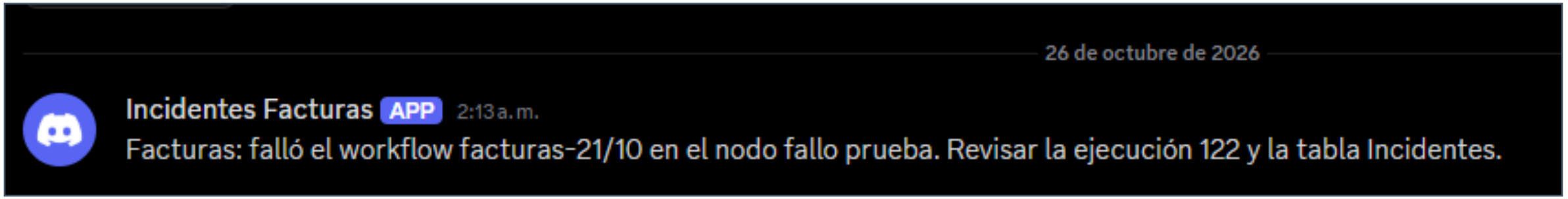
Pruebas realizadas

Datos ficticios · evidencias en documentacion/evidencias/

#	Prueba	Entrada ficticia	Esperado → observado	Tipo
1	**Caso normal**	C08-001, completa, 150000	200 y una fila nueva → cumplido	Manual
2	**Entrada incompleta**	Igual, sin cliente	400 y sin fila → cumplido	Manual
3	URL de producción	Factura válida, workflow publicado	Responde sin escucha manual → cumplido	Automática
4	Sin clave	Sin encabezado de clave	403 y sin fila → cumplido	Manual
5	Duplicada	Misma factura dos veces	200, luego 409; una fila → cumplido	Manual
6	**Fallo técnico** (tabla inexistente)	Factura válida, ID de tabla cambiado	502 ok:false e incidente → cumplido	Manual
7	**Fallo automático** + manejador	`forzar_error: true`	Ejecución en error; fila en Incidentes → cumplido	Automática
8	Recuperación	La factura que falló, reenviada	Una sola fila, sin duplicado → cumplido	Manual
9	Completa / incompleta	Dos facturas con distinto estado	Cada una guarda su estado → cumplido	Manual
10	Aviso en Discord	Fallo de la prueba 7	Mensaje en el canal → cumplido (ejecución 122)	Automática

Las pruebas 6 y 7 son fallos **simulados**, no caídas reales de Baserow: demuestran que la ruta de error funciona, no el comportamiento ante una caída real.

Evidencia: aviso en Discord



Qué demuestra esta captura

- El manejador de errores **sí envió** el aviso: el canal recibió el mensaje de la app «Incidentes Facturas» el 26 de octubre de 2026.
- El mensaje indica el workflow (`facturas-21/10`), el nodo donde falló (`fallo prueba`, el fallo controlado con `forzar_error`) y la ejecución (122).
- Invita a revisar la tabla **Incidentes**, donde queda el registro del fallo.
- Es un fallo **simulado**; no demuestra el comportamiento ante una caída real de Baserow.

Resultados, limitaciones y pendientes

Implementado y comprobado

- Entradas por formulario y Webhook con clave (pruebas 1 y 4).
- Validación de campos y control de duplicados (2 y 5).
- Subworkflow usado por el principal.
- Escritura en Facturas verificada en Baserow (1, 3, 8, 9).
- Aviso por Discord recibido en el canal (10).
- Ruta de error, incidente registrado y recuperación sin duplicar (6, 7, 8).

Simulado / sin evidencia

- Fallos provocados con tabla inexistente y `forzar_error`.
- Rama «Lista de prueba (4 facturas)»: práctica del primer corte con datos ficticios.

Propuesto (no implementado)

- Hacer único numero_factura en Baserow.
- Reintentos controlados con control de duplicados.
- Validar que el monto sea numérico y > 0.
- Validación y duplicados también en el formulario.
- Túnel cloudflared para recibir desde Internet.

Beneficios. No se midieron tiempos ni porcentajes de ahorro. Lo que se describe (menos digitación, menos duplicados, trazabilidad de fallos) son beneficios **esperados**; lo comprobado son las pruebas documentadas en esta presentación.

Acceso al proyecto

Repositorio de GitHub

```
https://github.com/JuanM330/Automatizacion_Facturas_n8n
```

Qué buscar	Dónde
Cómo instalar, configurar y probar	`README.md` (raíz)
Documentación completa del estado actual	`documentacion/Bitacora_Unificada_Proyecto.md`
Los tres workflows exportados	`workflows/` (principal, subworkflow, manejador)
Diagramas y evidencias de las pruebas	`documentacion/diagramas/` y `documentacion/evidencias/`
Bitácoras por clase (historial)	`documentacion/historial/`

Credenciales: cada persona crea las suyas (Baserow, clave del Webhook y Discord). No hay secretos en los archivos.